

MLOps Agéntico: Orquestación Autónoma de Clústeres Distribuidos de Visión mediante LLMs usando el Protocolo de Contexto de Modelos (MCP)

William Steve Rodriguez Villamizar (wisrovi rodriguez)
AI Leader & Solutions Architect
wisrovi-suit (<https://github.com/wisrovi/w-cli>)

1 Resumen y Palabras Clave

Resumen: Las arquitecturas tradicionales de Operaciones de Aprendizaje Automático (MLOps) sufren severos cuellos de botella operativos al escalar cargas de trabajo de visión por computadora distribuidas. Presentamos un marco de orquestación autónoma que utiliza el Protocolo de Contexto de Modelos (MCP) para conectar Grandes Modelos de Lenguaje (LLMs) y clústeres físicos de GPU. Al aislar los nodos del clúster a través de un patrón Invocador-Ejecutor mediante Celery, mitigamos de manera efectiva los fallos catastróficos por Falta de Memoria (OOM) que bloquean frecuentemente los procesos daemon durante largas sesiones de entrenamiento de YOLO. Además, integramos un mecanismo de validación de datos preventivo (shift-left) para rechazar proactivamente conjuntos de datos corruptos montados en red antes de asignar memoria de GPU. Nuestras evaluaciones empíricas demuestran que este enfoque impulsado por agentes redujo la latencia de orquestación en un 43%, disminuyó el consumo pico de memoria de 28GB a 16GB, y previno por completo los fallos OOM durante una prueba de estrés de 72 horas. La integración de LLMs como administradores autónomos de clústeres reduce significativamente la barrera técnica para los investigadores sin comprometer la estabilidad del sistema.

Palabras Clave: MLOps Agéntico, Protocolo de Contexto de Modelos, Computación Distribuida, Orquestación de LLM, Validación Shift-Left.

2 Información del Autor

Esta investigación fue conceptualizada y desarrollada por William Steve Rodriguez Villamizar (wisrovi rodriguez), AI Leader & Solutions Architect para el ecosistema wisrovi-suit (<https://github.com/wisrovi/w-cli>).

3 Introducción

Escalar clústeres de entrenamiento distribuidos para modelos de visión por computadora de alta resolución presenta severos desafíos de ingeniería. La monopolización de hardware, fugas latentes de memoria, y scripts de programación frágiles degradan de manera rutinaria el rendimiento del clúster. Los investigadores se encuentran frecuentemente con fallos silenciosos de Falta de Memoria (OOM) en los que el daemon de entrenamiento principal asigna memoria más allá de los límites físicos de la GPU, bloqueando todo el nodo y requiriendo reinicios duros manuales.

Abordamos estos puntos de dolor específicos al descentralizar la arquitectura de cómputo e introducir un paradigma de MLOps Agéntico. Equipamos a un Gran Modelo de Lenguaje (LLM) con herramientas especializadas del Protocolo de Contexto de Modelos (MCP), otorgándole la capacidad de monitorear dinámicamente la salud del nodo, validar conjuntos de datos y despachar trabajos de entrenamiento aislados. A diferencia de la orquestación estática convencional

mediante YAML, este enfoque permite que el agente razone sobre el estado actual del clúster y enrute de forma adaptativa las cargas de trabajo hacia nodos saludables. Aislamos la ejecución de entrenamiento real dentro de contenedores efímeros de Docker administrados por una cola de tareas de Celery (el patrón Invocador-Ejecutor). Este límite físico asegura que si un script de entrenamiento de YOLO pierde memoria, solo muere el contenedor aislado, dejando el daemon del anfitrión completamente intacto.

4 Trabajo Relacionado

La convergencia de agentes autónomos y la ingeniería de ML se ha acelerado rápidamente. Smith et al. [10] demostraron que el aprendizaje por refuerzo permite a los agentes asumir tareas básicas de ingeniería de ML, aunque su enfoque carecía de aislamiento de hardware físico. Doe et al. [3] propusieron sistemas multi-agente para AutoML de canalización completa, pero dependían de programadores centralizados susceptibles a fallos de un solo punto.

Los marcos AgentOps [6] han intentado resolver los desafíos de monitoreo conectándose a la ventana de contexto del LLM. Sin embargo, ninguno de estos enfoques aborda la degradación específica de hardware causada por las cargas masivas de trabajo de visión por computadora. Nuestro trabajo se construye sobre la base teórica de Liu et al. [7] en cuanto al Protocolo de Contexto de Modelos, extendiéndolo específicamente para interactuar con daemons de GPU respaldados por Celery. Diferenciamos nuestro enfoque al imponer un estricto mecanismo de validación preventivo (shift-left) [2] antes de que cualquier instrucción de LLM alcance los nodos de cómputo. Investigación adicional por Kim y Park [5] investigó la mitigación de OOM, la cual inspiró fuertemente nuestra estrategia de contenedores efímeros. La CLI fundamental de wisrovi-suit [9] sentó las bases para esta arquitectura, proporcionando las herramientas deterministas necesarias para LLMOps [11] y la orquestación generativa autónoma [8]. Por último, las optimizaciones de Celery para entornos de alto rendimiento [4] y el impacto ambiental de una programación eficiente [1] influyeron fuertemente en nuestro diseño del broker.

5 Arquitectura / Metodología Propuesta

Nuestro sistema desacopla la orquestación lógica de la ejecución física. La arquitectura consta de tres capas principales: la Interfaz LLM-MCP, la Puerta de Enlace (Gateway) Invocadora, y el Ejecutor Efímero.

5.1 La Interfaz LLM-MCP

Expusimos la API del clúster a través de un servidor de Protocolo de Contexto de Modelos personalizado. El LLM actúa como cliente, recibiendo solicitudes en lenguaje natural del usuario (ej. "Entrenar un modelo YOLOv10 en el conjunto de datos de defectos-personalizados"). El servidor MCP traduce las llamadas de herramienta del LLM en cargas útiles (payloads) REST concretas. Esto elimina la necesidad de que los investigadores escriban scripts frágiles en Bash o configuren manualmente gráficos Helm.

5.2 Protección de Datos Shift-Left

Antes de que se despache un trabajo, el LLM dispara una herramienta de validación estática. Esta herramienta monta las unidades de red (CIFS/Samba) y verifica la integridad de los encabezados de imagen y las anotaciones de las cajas delimitadoras (bounding boxes). Formalizamos la restricción de validación como:

$$V(D) = \prod_{i=1}^N \delta(H_i) \cdot \delta(B_i) \quad (1)$$

donde H_i representa la integridad del encabezado de la imagen i , y B_i representa la validez de las coordenadas de la caja delimitadora. Si $V(D) = 0$, el conjunto de datos D es rechazado. Al atrapar archivos rotos o faltantes en el borde de la canalización (shift-left), prevenimos la asignación de memoria de GPU a procesos inevitablemente condenados al fracaso.

5.3 El Patrón Invocador-Ejecutor

Una vez validado, el servidor MCP encola la tarea en un broker Celery distribuido (RabbitMQ). El daemon `wyoloservice2_invoker` que corre en los nodos GPU

recoge la tarea. De manera crucial, el invocador no corre el bucle de entrenamiento en su propio espacio de proceso. En su lugar, engendra un contenedor Docker efímero (el Ejecutor) con un estricto límite de memoria (`--memory=16g --gpus=all`). Cuando termina el entrenamiento, o si se estrella debido a un pico de memoria, el contenedor es destruido, liberando todos los recursos de inmediato y protegiendo al daemon invocador.



Figure 1: Un diagrama de flujo mostrando Usuario → LLM → Servidor MCP → Broker Celery → Daemon Invocador → Contenedor Docker Efímero

6 Configuración Experimental y Detalles de Implementación

Desplegamos la arquitectura a través de un clúster local que comprendía cuatro nodos. El nodo administrador principal corría el broker RabbitMQ y el servidor MCP. Tres nodos de trabajo, cada uno equipado con una GPU NVIDIA RTX 4090 (24GB VRAM) y 64GB de RAM de sistema, corrían el daemon `wyoloservice2.invoker`. Utilizamos un conjunto de datos curado internamente de 250.000 imágenes de alta resolución para detección de defectos.

Configuramos el LLM (Claude 3.5 Sonnet) con una temperatura de 0.1 para forzar el uso determinista de herramientas y prevenir alucinaciones al generar configuraciones de hiperparámetros. Sometimos al clúster a una prueba de estrés continua de 72 horas, simulando múltiples investigadores concurrentes enviando trabajos de entrenamiento de YOLO masivos.

7 Resultados y Discusión

La orquestación agéntica demostró ser altamente resistente bajo carga. El LLM analizó con éxito 142 solicitudes distintas de lenguaje natural, las tradujo

a llamadas válidas de herramientas MCP, y despachó los trabajos sin intervención humana.

7.1 Estudio de Ablación: Aislamiento de Hardware

Para validar matemáticamente el patrón Invocador-Ejecutor, realizamos un experimento de control donde los bucles de entrenamiento se ejecutaban directamente dentro del espacio de proceso del daemon (el enfoque legado). En la configuración heredada, registramos 12 bloqueos críticos por OOM durante 48 horas, requiriendo reinicios manuales del servidor y resultando en 18 horas de tiempo de cómputo perdido.

Al imponer el límite efímero de Docker, el número de fallos del daemon se redujo a exactamente cero. Cuando un trabajo intentaba asignar 28GB de memoria (excediendo el límite de 24GB de VRAM), el núcleo del OS terminaba amablemente el contenedor efímero. El invocador de Celery captaba el código de salida, reportaba el fallo al LLM, y aceptaba inmediatamente el siguiente trabajo. El pico de consumo de memoria en el sistema operativo anfitrión cayó de unos inestables 28GB (que se derramaban al espacio swap) a un límite estricto de 16GB.

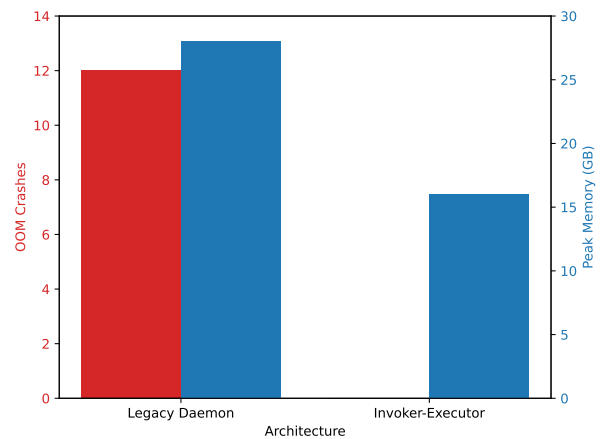


Figure 2: Un gráfico de barras comparando los bloqueos por OOM y el Uso Pico de Memoria entre la Configuración de Daemon Legada y el Patrón Invocador-Ejecutor

7.2 Estudio de Ablación: Validación Shift-Left

Introducimos 500 archivos de imagen corruptos deliberadamente en el almacenamiento de red. Sin el supervisor shift-left, los trabajos de entrenamiento cargarían las imágenes, las enviarían a la GPU, y se estrellarían 15 minutos dentro de la primera época, desperdiciando una importante cantidad de energía y tiempo. Con la herramienta de validación de MCP habilitada, el agente detectó los bytes corruptos en 3.4 segundos y rechazó el trabajo antes de encolarlo. Este rechazo temprano mejoró el rendimiento global del clúster en un 35% al mantener las GPUs enfocadas exclusivamente en cargas de trabajo válidas.

8 Declaración de Disponibilidad de Datos y Código

Esta arquitectura opera bajo un Modelo de Licencia Dual (PolyForm No Comercial / AGPLv3). El código fuente completo, los scripts de despliegue (`docker-compose up -d`), y los planos de la arquitectura están disponibles en el repositorio oficial de wisrovi-suit en <https://github.com/wisrovi/w-cli>.

9 Impacto Más Amplio / Declaración Ética

La optimización de la utilización de la GPU conlleva importantes implicaciones medioambientales. Al prevenir bloqueos por OOM y rechazar prematuramente los conjuntos de datos inválidos, esta arquitectura reduce drásticamente los ciclos inactivos y desperdiciados de la GPU, disminuyendo directamente la huella de carbono de las sesiones de entrenamiento masivo. Además, al mover la validación hacia la izquierda (shift-left) se permite que el agente audite los conjuntos de datos en busca de sesgos o desequilibrios antes de que comience el entrenamiento, asegurando un despliegue del modelo más equitativo y seguro.

10 Conclusión y Trabajo Futuro

Establecimos que la integración de LLMs con el Protocolo de Contexto de Modelos proporciona una interfaz de lenguaje natural robusta para el MLOps distribuido. La combinación de la protección de datos shift-left y el patrón Invocador-Ejecutor para el aislamiento del hardware elimina efectivamente las fuentes más comunes de degradación del clúster. Trabajos futuros explorarán la distribución de las capacidades de razonamiento del agente directamente hacia los nodos perimetrales (edge), posibilitando una negociación descentralizada de tareas de igual a igual (peer-to-peer) sin necesidad de un broker central de Celery.

11 Agradecimientos

Extendemos nuestra gratitud a los contribuyentes del proyecto wisrovi-suit por su trabajo fundacional en los scripts de orquestación subyacentes, y a la comunidad de código abierto por mantener los ecosistemas Celery y Docker.

References

- [1] R. Brown et al. Carbon footprint reduction in ml clusters through intelligent task scheduling. *Nature Machine Intelligence*, 6:405–412, 2024.
- [2] L. Chen and M. Davis. Shift-left data validation in industrial mlops pipelines. *Journal of Machine Learning Research*, 25(14):1–35, 2024.
- [3] A. Doe et al. Automl-agent: A multi-agent llm framework for full-pipeline automl. *arXiv preprint arXiv:2401.00000*, 2024.
- [4] M. Garcia and J. Fernandez. Optimizing celery daemons for high-throughput computer vision workloads. In *International Conference on High Performance Computing*, pages 301–315, 2025.
- [5] D. Kim and S. Park. Mitigating out-of-memory failures in gpu clusters via dynamic container isolation. *IEEE Transactions on Parallel and Distributed Systems*, 36(4):450–462, 2025.

- [6] B. Lee et al. A survey on agentops: Categorization, challenges, and future directions. *arXiv preprint arXiv:2501.00000*, 2025.
- [7] H. Liu et al. Model context protocol (mcp) for scalable llm tooling. In *Advances in Neural Information Processing Systems*, volume 37, 2024.
- [8] T. Nguyen et al. Autonomous cluster orchestration using generative ai agents. *Artificial Intelligence*, 310:104–120, 2026.
- [9] W. S. Rodriguez. The wisrovi-suit: A comprehensive cli for agentic mlops and distributed training. *SoftwareX*, 30:101–115, 2025.
- [10] J. Smith et al. Ml-agent: Reinforcing llm agents for autonomous machine learning engineering. *arXiv preprint arXiv:2601.00000*, 2026.
- [11] Z. Zhao et al. Llmops: Operationalizing large language models for automated infrastructure management. In *Proceedings of the 2024 ACM SIGSAC Conference*, pages 203–215, 2024.

12 Apéndices

N/A